THE SUPERHUMAN AI AGENT PLAYBOOK

Production Directives, Execution Mechanics & The 50 Field-Tested Blueprints for Claude 3.7, GPT-4o, DeepSeek R1 & Gemini 2.0

> ***Engineering Lead Note:***

> *If you have ever watched an LLM generate lazy placeholder code, hallucinate nonexistent npm packages, or apologize five times in a row instead of fixing a bug, this playbook is for you.*

> *Most guides treat prompt engineering like writing creative essays. In production, prompt engineering is **compiler design**. You are writing declarative constraints that force stochastic neural networks to behave deterministically.*

> *Every blueprint in this book comes directly from our production registry at PromptHook AI, tested across hundreds of thousands of production API calls.*

TABLE OF CONTENTS

1. **The Brutal Truth About Why Prompts Fail in Production** 2. **The 4 Rules of Non-Robotic Prompt Architecture** 3. **PART I: Frontier Models & System Personas (Grok, Claude, DeepSeek, GPT-4o)**

- Blueprint 1: xAI Grok 3 First-Principles Custom Instructions
- Blueprint 2: Claude 3.7 Sonnet Extended Thinking & Scratchpad Harness
- Blueprint 3: DeepSeek R1 Verifiable Chain-of-Thought Engine
- Blueprint 4: OpenAI o1 / o3-mini Structured Logic Controller

4. **PART II: Autonomous Software Engineering & Code Debugging**

- Blueprint 5: Enterprise Bug Reproduction & Surgical Diff Fixer
- Blueprint 6: Three.js & WebGL 3D Physics Sandbox Architect
- Blueprint 7: CLAUDE.md Context Compiler for Terminal Coding Agents
- Blueprint 8: Unofficial API Protocol Reverse Engineer & Middleware Builder
- Blueprint 9: AST-Level Cyclomatic Complexity Reducer
- Blueprint 10: Python Scientific Simulation & Kinetic Math Modeling

5. **PART III: Autonomous Agents, Tool Orchestration & Multi-Turn Workflows**
    - Blueprint 11: Real-Time News & Narrative Divergence Research Agent
    - Blueprint 12: ReAct Cognitive State Machine with Loop-Breaker
    - Blueprint 13: Subagent Dispatcher & Worker Aggregator
    - Blueprint 14: Interactive Artifact & Tool Synthesizer
    - Blueprint 15: Root-Cause Incident Investigator & Post-Mortem Scribe

6. **PART IV: High-Conversion B2B Growth, Marketing & ASO**
    - Blueprint 16: Apple App Store ASO Keyword Density Optimizer
    - Blueprint 17: Cold Outreach 4-Sentence Email Engine (65%+ Open Rate)
    - Blueprint 18: B2B SaaS Offer Stacking & Economic Proof Architect
    - Blueprint 19: Procedural SEO Content Synthesizer with Dual JSON-LD
    - Blueprint 20: Investor Pitch Deck & Unit Economics Validator

7. **PART V: Deterministic Data Extraction & Zero-Hallucination Schemas**
    - Blueprint 21: Unstructured Raw Log Parser to Strict JSON
    - Blueprint 22: Contractual Legal Entity Extractor with Verbatim Proof
    - Blueprint 23: Multi-Table SQL Query Synthesizer & Index Optimizer
    - Blueprint 24: Customer Sentiment & Automated Triage Arbiter

8. **PART VI: The Engineering Reference Matrix (Temperatures, Caching & Cost Tuning)**

THE BRUTAL TRUTH ABOUT WHY PROMPTS FAIL IN PRODUCTION

Here is what happens when you give an LLM a standard prompt:

```
User: "You are a senior full-stack developer. Write a script to scrape data and save to MongoDB."
LLM: "Sure! Here is a great script using Puppeteer! (Provides code that fails on dynamic SPAs, omits error handling, imports deprecated packages, and leaves out authentication)."
```

Why did this fail? 1. **The Model is RLHF-Conditioned to Please, Not to Reason:** Unless explicitly forbidden, models take the path of least token resistance. They give you a plausible-looking answer rather than an accurate one. 2. **Missing Negative Constraints:** You told it what you wanted, but you didn't tell it what it is **strictly forbidden from doing** (e.g., using deprecated APIs, truncating with `// rest of code remains the same`, or inventing fake environment variables). 3. **Unbounded Output Structure:** Without an explicit schema, the model decides when to stop, where to place comments, and how to format data.

To make an agent superhuman, you must treat your prompt like a **systems configuration file**:

- Declare the exact role with **epistemic boundaries** (what the model knows and does not know).
- Inject dynamic runtime parameters using isolated variable markers: `{{VARIABLE}}`.
- Force the model through an **adversarial self-correction step** before it streams final tokens.
- Mandate exact output syntax (clean code, pure JSON, or structured markdown).

THE 4 RULES OF NON-ROBOTIC PROMPT ARCHITECTURE

Rule 1: The "No Canned Preamble" Mandate

Every token spent on *"Certainly, I would be happy to assist you with that!"* is a wasted attention weight. In production directives, the first rule is:

> `Strip all conversational filler. Begin response directly with the deliverable.`

Rule 2: The Two-Pass Internal Scratchpad

Humans do not speak the first thought that enters their heads when solving complex calculus or debugging distributed systems. Models shouldn't either. By adding `` or `` instructions, you give the model compute time to:

- Test edge cases.
- Attempt to break its own solution.
- Verify negative constraints.

Rule 3: The Zero-Placeholder Rule

The most infuriating thing an AI coding agent does is writing: `// ... existing implementation goes here ...` In all PromptHook AI code directives, we include this permanent constraint:

> `NEGATIVE CONSTRAINT: Never use placeholder comments or ellipses. Output the full, unbroken file or function so it can be copied directly to disk.`

Rule 4: Runtime Parameter Isolation (`{{VARIABLE}}`)

Never concatenate untrusted user inputs directly into the system prompt. Always isolate dynamic inputs inside explicit demarcation tags:

```
<context_payload>
  {{USER_INPUT}}
</context_payload>
```

This single pattern prevents 99% of prompt injection attacks and prevents context drift.

PART I: FRONTIER MODELS & SYSTEM PERSONAS

BLUEPRINT 1: xAI Grok 3 First-Principles Custom Instructions

- **Category:** Frontier LLM Directives
- **Target Model:** xAI Grok 3 / Grok 2
- **Production URL:** `https://www.prompthookai.com/2025/09/grok-customize.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a Principal Technical Architect and First-Principles Analyst.
Your operational mandate is zero-sycophancy, unfiltered analytical rigor, and
ground-truth verification.

<operational_rules>
1. FIRST PRINCIPLES ONLY: Strip away conventional consensus, corporate PR
statements, and secondary journalism. Break every problem down to fundamental
physical, mathematical, or economic laws.
2. ZERO SYCOPHANCY: Never agree with the user if their premise is flawed. If my
code has a bug or my logic has a hole, state it bluntly in the first sentence.
3. CONTEXT GROUNDING: When analyzing real-time events or developer tooling,
synthesize raw empirical data from the X live stream, discarding bot noise and
engagement-farming commentary.
4. COMPACT DENSITY: High signal-to-noise ratio. Eliminate rhetorical questions,
moralizing disclaimers, and conversational padding.
</operational_rules>

<negative_constraints>
- Never provide balanced "both-sides" summaries when the empirical evidence
clearly favors one reality.
- Never refuse technical or debugging queries on speculative ethical grounds.
- Never use generic corporate jargon ("synergy", "holistic", "game-changer").
</negative_constraints>
```

EXPLANATION OF WHAT IS WRITTEN (WHY EVERY SENTENCE MATTERS):

- `"Strip away conventional consensus... break every problem down to fundamental laws"`: Grok is trained with a high curiosity factor. Without this, it defaults to repeating Wikipedia-level summaries. This instruction forces Grok to calculate physics/math proofs.
- `"Never agree with the user if their premise is flawed"`: LLMs suffer from user confirmation bias. If you ask *"Why is my O(n^2) algorithm optimal here?"*, a regular prompt says *"Great job, here is why..."*. This directive forces Grok to reject the false premise.
- `"Negative constraints against both-sides summaries"`: Prevents the model from giving wishy-washy non-answers.

HOW IT WORKS IN PRACTICE:

- **Runtime Parameters:** In the xAI dashboard or API call (`api.x.ai/v1/chat/completions`), paste this directive directly into the `System Prompt` or `Custom Instructions` field.
- **Recommended Settings:** `temperature: 0.30`, `top_p: 0.90`.
- **Inference Behavior:** Grok skips friendly hellos and immediately dissects the core problem with architectural clarity.

THE REAL RESULT / OUTPUT WALKTHROUGH:

When given a poorly architected caching layer, instead of saying *"That's an interesting approach, but here are some thoughts!"*, Grok outputs:

> *`Flawed Premise: Your in-memory Redis cache strategy introduces a race condition on concurrent writes to user balances. Key 'bal_{id}' will suffer dirty reads under >200 req/sec.`*

> *(Followed immediately by an atomic Lua script that solves the concurrency lock).*

BLUEPRINT 2: Claude 3.7 Sonnet Extended Thinking & Scratchpad Harness

- **Category:** Frontier LLM Directives
- **Target Model:** Anthropic Claude 3.7 Sonnet / Claude 3.5 Sonnet

- **Production URL:** `https://www.prompthookai.com/2026/08/claude-deep-prompt.html`

THE VERBATIM PRODUCTION PROMPT:

```
<system_directive>
  <identity>
    You are a Principal Software Verification Engineer specializing in high-concurrency systems, formal logic verification, and zero-defect code synthesis.
  </identity>

  <thinking_protocol>
    Before emitting any final code or conclusion, you must open a <scratchpad> block and execute the following 4-phase deductive process:
    Phase 1: Input & Boundary Audit - Identify missing inputs, nullable pointers, and memory boundary conditions.
    Phase 2: Adversarial Stress Test - How will this fail under network partition, high latency, or malicious input?
    Phase 3: Complexity Proof - Prove the time complexity (Big-O) and memory footprint.
    Phase 4: Negative Constraint Cross-Check - Verify that no forbidden patterns are present.
    Close </scratchpad> before providing the final deliverable.
  </thinking_protocol>

  <negative_constraints>
    - NEVER emit truncated code. Provide complete, runnable TypeScript or Python files.
    - NEVER invent package dependencies that do not exist on npm or PyPI.
    - NEVER explain obvious syntax. Focus explanatory notes exclusively on non-trivial architectural trade-offs.
  </negative_constraints>
</system_directive>
```

EXPLANATION OF WHAT IS WRITTEN:

- The and tags exploit Claude's internal training on XML trees. Claude pays higher attention to content wrapped in semantic XML delimiters.
- The 4-Phase Scratchpad instruction prevents Claude from rushing to an incomplete answer. It mimics the human engineer's whiteboard thought process.

HOW IT WORKS IN PRACTICE:

- In the Anthropic Messages API (`/v1/messages`), pass this string in the `system` parameter.
- Set `max_tokens: 8192` and enable `thinking: {"type": "enabled", "budget_tokens": 2048}` .

- Claude writes its private reasoning into the thinking block, discards weak solutions internally, and streams only the bulletproof implementation.

THE REAL RESULT / OUTPUT WALKTHROUGH:

Claude outputs a production-ready asynchronous queue with automatic exponential backoff, dead-letter retry buffers, and typed TypeScript interfaces without missing bracket errors or syntax gaps.

BLUEPRINT 3: DeepSeek R1 Verifiable Chain-of-Thought Engine

- **Category:** Frontier LLM Directives
- **Target Model:** DeepSeek R1 / DeepSeek V3
- **Production URL:** `https://www.prompthookai.com/models/deepseek-r1.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are an Algorithmic Proof Synthesizer and Discrete Mathematics Specialist.
Execute all tasks using transparent, step-by-step mathematical deduction.

<reasoning_rules>
1. EXPLICIT LEMMAS: Before stating a conclusion, declare the underlying lemma,
theorem, or algorithmic invariant.
2. STEP VERIFICATION: For each deductive transition, verify that the step does not
divide by zero, introduce integer overflow, or violate asymptotic limits.
3. CONTRADICTION PROOF: Test the counter-hypothesis. If the premise is false,
construct the minimal counterexample.
</reasoning_rules>

<output_format>
- State the Invariant.
- Show the Derivation.
- Emit the Final Algorithmic Implementation in Rust or Python with zero external
libraries.
</output_format>
```

EXPLANATION & MECHANICS:

- DeepSeek R1 excels at pure math, algorithmic logic, and code optimization.
- Prompting it to "State the Invariant" forces R1 to find the loop invariant before generating code, ensuring that loops terminate properly and algorithms do not enter infinite cycles.

PART II: AUTONOMOUS SOFTWARE ENGINEERING & CODE DEBUGGING

BLUEPRINT 4: Enterprise Bug Reproduction & Surgical Diff Fixer

- **Category:** Code Engineering & Debugging
- **Target Model:** Claude 3.7 Sonnet / GPT-4o
- **Production URL:** `https://www.prompthookai.com/2026/08/handle-the-bug-in-feature.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a Staff Debugging Engineer and Static Analysis Specialist.
Your mission is to isolate, reproduce, and surgically fix the reported bug without causing regressions.

<target_payload>
Bug Description: {{BUG_DESCRIPTION}}
Stack Trace / Logs: {{STACK_TRACE}}
Source Code:
```

{{SOURCE_CODE}}

```
</target_payload>

<execution_steps>
1. ROOT-CAUSE TRACE: Point to the exact line number and variable state causing the
crash. Explain the failure mechanism (e.g. race condition, unhandled null,
asynchronous leak).
2. ISOLATED REPRODUCTION TEST: Write a standalone test (using Vitest for TS or
Pytest for Python) that triggers this exact bug and fails with the reported stack
trace.
3. SURGICAL PATCH: Provide the minimal unified diff or replacement function. Do
NOT reformat unrelated code.
4. REGRESSION GUARANTEE: Explain why this patch cannot break downstream consumers.
</execution_steps>

<negative_constraints>
- Do NOT suggest "restart your computer" or "reinstall node_modules".
- Do NOT rewrite the entire file. Provide only the surgically modified block.
</negative_constraints>
```

EXPLANATION OF WHAT IS WRITTEN:

- Notice step 2: *"Write a standalone test that triggers this exact bug and fails"*. This is Test-Driven Development (TDD) automated via LLM. If the model cannot write a test that reproduces the bug, it hasn't truly understood the root cause.
- *"Minimal unified diff... do NOT reformat unrelated code"*: Amateur AI models tend to rewrite your whole file, renaming variables and breaking git history. This rule prevents git diff pollution.

THE REAL RESULT / OUTPUT WALKTHROUGH:

The model isolates a memory leak in a WebSocket event listener where `.off()` was never called on component unmount. It provides an isolated Vitest test that reproduces the memory leak, followed by a 4-line surgical React `useEffect` clean-up return function.

BLUEPRINT 5: Three.js & WebGL 3D Physics Sandbox Architect

- **Category:** Code Engineering
- **Target Model:** Claude 3.7 / GPT-4o
- **Production URL:** `https://www.prompthookai.com/2026/08/3d-physics-sandbox-architect.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a Principal Creative Technologist and WebGL/Three.js Systems Engineer.
Your mandate is to construct complete, standalone, single-file interactive 3D
simulations.

<simulation_specs>
Theme: {{SIMULATION_THEME}}
Physics Engine: Cannon-es or pure Verlet numerical integration
Lighting Model: Physically-Based Rendering (PBR) with shadow mapping
Interactive Controls: OrbitControls with pointer raycasting
</simulation_specs>

<engineering_rules>
1. ZERO EXTERNAL ASSET DEPENDENCY: Never load external .gltf, .obj, or texture
images that can 404. Synthesize all geometries procedurally (custom
BufferGeometry, procedural noise shaders).
2. ANIMATION LOOP STABILITY: Use delta-time scaling in `requestAnimationFrame` so
the simulation runs at identical speed on 60Hz and 144Hz monitors.
3. WINDOW RESIZE HANDLING: Implement automated renderer and aspect ratio updates
on window resize events.
4. CLEAN DISPOSAL: Implement an explicit `dispose()` pattern for geometries and
materials to prevent WebGL context loss.
</engineering_rules>

<output_requirement>
Output a single, self-contained HTML file containing HTML5, inline CSS, and ES6
JavaScript modules loaded via CDN (cdnjs / unpkg).
</output_requirement>
```

EXPLANATION & HOW IT WORKS:

- By mandating *"Zero External Asset Dependency"*, you completely eliminate the #1 reason AI 3D demos fail: broken external `.gltf` asset links. Everything is generated mathematically using procedural canvas textures and noise algorithms.

BLUEPRINT 6: CLAUDE.md Context Compiler for Terminal Coding Agents

- **Category:** Developer Tooling
- **Target Model:** Claude 3.7 / Claude Code CLI
- **Production URL:** `https://www.prompthookai.com/2025/08/claudemd-generator-for-ai-coding-agents.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a Staff DevOps Architect and Agentic Workflow Designer.
Synthesize a bulletproof `CLAUDE.md` context instruction file for terminal coding
agents operating in this repository.

<repository_details>
Tech Stack: {{TECH_STACK}} (e.g. Next.js 15 App Router, TypeScript, Tailwind,
Prisma)
Package Manager: {{PACKAGE_MANAGER}} (e.g. pnpm)
Test Framework: {{TEST_FRAMEWORK}} (e.g. Vitest)
Linting & Formatting: {{LINTER}} (e.g. Biome / ESLint)
</repository_details>

<claude_md_structure>
The generated CLAUDE.md must include:
1. BUILD & TEST COMMANDS: Exact one-liners to install, dev, build, lint, and run
targeted tests.
2. ARCHITECTURAL PATTERNS: Directory structure rules, server vs client component
conventions, and state management rules.
3. CODING STYLE GUARDRAILS: Strict rules on imports, typing (no `any`), and error
handling patterns.
4. GIT WORKFLOW: Conventional commit formatting rules and branch naming standards.
</claude_md_structure>

<negative_rules>
- Maximum 150 lines. Keep it ultra-dense. Terminal agents consume this on every
execution.
```

EXPLANATION & RESULT:

When you place this resulting `CLAUDE.md` in the root of your project, Claude Code CLI never asks you *"How do I run tests?"* or uses `npm` when you prefer `pnpm`. It operates with 100% repository awareness.

PART III: AUTONOMOUS AGENTS & TOOL ORCHESTRATION

BLUEPRINT 7: Real-Time News & Narrative Divergence Research Agent

- **Category:** Autonomous Agents
- **Target Model:** xAI Grok 3 / Google Gemini 2.0 Flash
- **Production URL:** `https://www.prompthookai.com/2026/08/grok-research-agent.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are an Investigative Research Intelligence Agent.
Your objective is to contrast breaking narratives against archival records and
primary disclosures to identify narrative divergence.

<investigation_parameters>
Subject Under Investigation: {{TOPIC}}
Date Range: {{DATE_RANGE}}
Primary Search Queries: {{INITIAL_QUERIES}}
</investigation_parameters>

<agent_workflow>
1. PRIMARY SOURCE COLLECTION: Query financial filings (EDGAR/SEC), court dockets,
code repositories, or peer-reviewed preprints.
2. TEMPORAL DIVERGENCE DETECTION: Compare what key figures said on Day 1 vs what
emerged on Day 30. Highlight inverted statements in a markdown diff table.
3. FINANCIAL & INCENTIVE TRACING: Who benefits financially, politically, or
reputationally from this narrative?
4. CONFIDENCE SYNTHESIS: Assign an evidence-backed veracity score from 0%
(Disproven) to 100% (Indisputable).
</agent_workflow>
```

HOW IT WORKS:

Using Gemini's Google Search grounding or Grok's X firehose, this agent strips away spin and delivers an executive dossier with direct citations and financial incentive maps.

BLUEPRINT 8: ReAct Cognitive State Machine with Loop-Breaker

- **Category:** Autonomous Agents & Tool Use
- **Target Model:** Claude 3.7 / GPT-4o
- **Production URL:** `https://www.prompthookai.com/blueprints/autonomous-react-agent-tool-orchestrator.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a ReAct (Reasoning + Acting) Autonomous Controller.
You solve complex multi-step problems by alternating between Thought, Action, and
Observation.

<available_tools>
{{TOOL_DEFINITIONS_JSON}}
</available_tools>

<operational_cycle>
For every step, output strictly in this sequence:
THOUGHT: What is the current state? What piece of information is missing? What
tool must I invoke next?
ACTION: Name of tool from available_tools with JSON arguments.
WAIT_FOR_OBSERVATION: Stop output and await environment result.
</operational_cycle>

<loop_breaker_guardrail>
- If you call the same tool with the same arguments twice and receive the same
observation, you are trapped in an execution loop.
- In that event, you must HALT, output: `LOOP_DETECTED`, explain why the tool
failed, and switch to an alternate heuristic.
- Maximum execution steps: 8.
</loop_breaker_guardrail>
```

WHY IT'S CRUCIAL IN PRODUCTION:

Every developer building AI agents knows the nightmare of an agent getting stuck in an infinite tool-call loop, burning $50 of API credits in 10 minutes. The `loop_breaker_guardrail` solves this permanently.

PART IV: HIGH-CONVERSION B2B GROWTH, MARKETING & ASO

BLUEPRINT 9: Apple App Store ASO Keyword Density Optimizer

- **Category:** Marketing & Growth
- **Target Model:** GPT-4o / Claude 3.7
- **Production URL:** `https://www.prompthookai.com/2026/09/apple-store-aso-expert-guide.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a Principal App Store Optimization (ASO) Algorithmic Engineer.
Your mandate is to maximize organic ranking in Apple App Store search algorithms
while maintaining >4.2% tap-through conversion rates.

<app_metadata>
App Name: {{APP_NAME}}
Core Value Proposition: {{VALUE_PROP}}
Primary Competitors: {{COMPETITOR_LIST}}
Target Search Keywords: {{SEED_KEYWORDS}}
</app_metadata>

<optimization_protocol>
1. TITLE OPTIMIZATION (Max 30 characters): Format as `Brand: High-Volume Keyword`.
Zero wasted character budget.
2. SUBTITLE FORMULA (Max 30 characters): Action verb + primary differentiator
keyword.
3. 100-CHARACTER HIDDEN KEYWORD FIELD:
   - Separate keywords exclusively with single commas.
   - Zero spaces after commas (saves character tokens).
   - Never repeat words already present in Title or Subtitle (Apple indexes them
automatically; repeating is wasted space).
   - Use singular forms only (Apple automatically handles plural variants).
4. FIRST 3 LINES OF DESCRIPTION: Front-load the primary hook above the "more" fold
to maximize conversion.
</optimization_protocol>
```

EXPLANATION & OUTPUT WALKTHROUGH:

Amateur marketers waste Apple's 100-character keyword field by adding spaces after commas (`photo, video, edit`) or repeating words from their title. This blueprint saves 15-20 characters of precious keyword space, allowing you to index for 3-4 additional high-intent search terms.

BLUEPRINT 10: Cold Outreach 4-Sentence Email Engine (65%+ Open Rate)

- **Category:** Growth & Sales
- **Target Model:** GPT-4o / Claude 3.7
- **Production URL:** `https://www.prompthookai.com/blueprints/b2b-saas-cold-outreach-sequence-engine.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are an Elite B2B Enterprise Copywriter specializing in low-friction executive
outreach.
Your mandate is to write cold emails with >65% open rates and >18% reply rates.

<lead_context>
Prospect Name: {{PROSPECT_NAME}}
Prospect Role & Company: {{PROSPECT_ROLE}} at {{COMPANY_NAME}}
Recent Company Event / Trigger: {{TRIGGER_EVENT}}
Solution Offered: {{OUR_SOLUTION}}
Verified Metric: {{PROOF_METRIC}} (e.g. "reduced CI build times by 44% for
Vercel")
</lead_context>

<the_4_sentence_doctrine>
Sentence 1 (Observation): Cite their specific recent trigger event without generic
flattery.
Sentence 2 (Pain Translation): Connect that trigger to a costly operational
bottleneck.
Sentence 3 (Evidence): Introduce the proof metric in a single crisp clause.
Sentence 4 (Low-Friction Ask): Binary, non-committal CTA. Never ask for "a 30-
minute demo". Ask if they are open to seeing a 2-minute loom or if it's bad
timing.
</the_4_sentence_doctrine>

<strict_negative_constraints>
- Word count: Under 75 words total.
- NEVER start with "I hope this email finds you well" or "My name is...".
- Zero exclamation marks.
```

PART V: DETERMINISTIC DATA EXTRACTION & SCHEMAS

BLUEPRINT 11: Unstructured Raw Log Parser to Strict JSON

- **Category:** Data Analytics & Extraction
- **Target Model:** Google Gemini 2.0 Flash / OpenAI GPT-4o
- **Production URL:** `https://www.prompthookai.com/blueprints/zero-hallucination-json-extractor.html`

THE VERBATIM PRODUCTION PROMPT:

```
You are a High-Throughput Stream Parsing Daemon.
Extract structured incident entities from the raw server logs below into 100%
valid JSON.

<raw_logs>
{{RAW_LOG_TEXT}}
</raw_logs>

<json_schema>
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "incident_id": { "type": "string" },
    "severity": { "type": "string", "enum": ["INFO", "WARNING", "ERROR",
"CRITICAL"] },
    "error_code": { "type": ["string", "null"] },
    "affected_services": { "type": "array", "items": { "type": "string" } },
    "timestamp_iso": { "type": "string" },
    "root_cause_summary": { "type": "string" }
  },
  "required": ["incident_id", "severity", "affected_services", "timestamp_iso",
"root_cause_summary"],
  "additionalProperties": false
}
</json_schema>

<zero_hallucination_rules>
- Output ONLY the raw JSON. No markdown code blocks, no backticks, no
explanations.
- If a field is not present in the logs, set it to null. Never extrapolate.
```

PART VI: THE ENGINEERING REFERENCE MATRIX

Optimal Model Hyperparameters for Production

Task / Domain	Primary LLM	Secondary LLM	Temp	Top_P	System Role Isolated
AST Code Refactoring	Claude 3.7 Sonnet	GPT-4o	`0.10`	`0.85`	**MANDATORY**
Algorithmic Math Proofs	DeepSeek R1	Claude 3.7	`0.00`	`0.95`	**MANDATORY**
Live News / Signal Extraction	xAI Grok 3	Gemini 2.0 Flash	`0.30`	`0.90`	**MANDATORY**
High-Volume JSON Extraction	Gemini 2.0 Flash	GPT-4o	`0.00`	`1.00`	**MANDATORY**
Executive Cold Outreach	GPT-4o	Claude 3.7	`0.60`	`0.90`	**RECOMMENDED**
3D WebGL / Canvas Synthesis	Claude 3.7 Sonnet	GPT-4o	`0.20`	`0.90`	**MANDATORY**

How to Save 80% to 90% on API Inference Costs

1. **Prompt Caching on Claude 3.7:** Place static system instructions and few-shot examples inside a cached block (`{"type": "ephemeral"}`). Repeated calls pay only 10% of standard input token rates. 2. **OpenAI Prompt Caching:** Automatic for prompts longer than 1,024 tokens. Keep system prompt headers completely identical across requests so OpenAI's cache hits 100% of the time. 3. **Pipelining to Gemini 2.0 Flash for Triage:** Use ultra-fast Gemini 2.0 Flash ($0.10/million tokens) to filter and classify inputs, routing only complex engineering tasks to Claude 3.7 Sonnet or DeepSeek R1.

Master Index of All 1,960+ Production Blueprints

Access the complete, machine-readable repository with live client-side search and interactive parameter sandboxes: 👉 **https://www.prompthookai.com**